

# **Information Technologies for Industrial Engineers**

# Vue Basics

# Why Vue?

- Easy to start: write mostly HTML, CSS, and JavaScript.
- Clear syntax for beginners.
- Great for building interactive web pages.

# Vue App Structure

main.js

```
import { createApp } from "vue";  
import App from "./App.vue";  
  
createApp(App).mount("#app");
```

- `createApp(...)` creates your app.
- `.mount("#app")` tells Vue where to render on the page.

# Template Syntax

## App.vue

```
<script setup lang="ts">
const msg = "Hello World";
const num = 1;
const birthYear = 2000;
const currentYear = new Date().getFullYear();
</script>

<template>
  <div>This is just like HTML.</div>
  <div>{{ 1 + 1 }}</div>
  <div>{{ msg }}</div>
  <div>{{ num + 1 }}</div>
  <div>I am {{ currentYear - birthYear }} years old.</div>
</template>
```

# Style

App.vue

```
<template>
  <div class="wrapper">
    <h2 class="header">Test CSS</h2>
  </div>
</template>

<style scoped>
.wrapper {
  background-color: lightgray;
}
.header {
  color: teal;
}
</style>
```



# Event Handling

App.vue

```
<script setup lang="ts">
function handleClick() {
  alert("Hello!");
}
</script>

<template>
  <button @click="handleClick">Click</button>
</template>
```

- `@click` listens for click events.
- Other events: `@mouseover`, `@keydown`, `@submit`, etc.

# Component

- Components are reusable UI blocks.
- Each component has its own template, script, and style.

## components/NavBar.vue

```
<template>
  <div class="nav">
    <span>Home</span>
    <span>About Me</span>
  </div>
</template>

<style scoped>
.nav {
  display: flex;
  gap: 0.5em;
  background-color: lightgray;
  padding: 0.2em;
}
</style>
```

App.vue

```
<script setup>
import NavBar from "./Navbar.vue";
</script>

<template>
  <NavBar />
  <h1>Welcome to My App</h1>
</template>
```

# State

# Counter App (V1)

```
<script setup lang="ts">
let counter = 0;
function handleClick() {
  counter += 1;
  console.log(counter);
}
</script>

<template>
  <div>{{ counter }}</div>
  <button @click="handleClick">Add</button>
</template>
```

- `@click` is a **directive** that listens for click events.



# Counter App (V1)

- Why doesn't it work?
- If you `console.log counter`, it will show the correct value, but the template doesn't update.

The reason is that Vue doesn't know that `counter` has changed, so it doesn't re-render the template.

## ref(...)

- Use `ref(...)` to create **reactive state**.
- `ref(...)` returns an object with a `.value` property.
- In template, you can use the `ref` directly without `.value`.
- In JavaScript, you must use `.value` to access the value.

# Counter App (V2)

```
<script setup lang="ts">
import { ref } from "vue";

const counter = ref(0);

function handleClick() {
  counter.value += 1;
}
</script>

<template>
  <div>{{ counter }}</div>
  <button @click="handleClick">Add</button>
</template>
```

# Common Mistake

- In JavaScript, use `counter.value`.
- In template, use `{{ counter }}`.

# Form V1

```
<script setup>
import { ref } from "vue";

const name = ref("");
function handleInput(e) {
  name.value = e.target.value;
}

function handleClick() {
  name.value = "";
}
</script>

<template>
  <input @input="handleInput" :value="name" placeholder="Type your name" />
  <p>Hello, {{ name }}</p>
  <button @click="handleClick">Submit</button>
</template>
```

# Directives

- `@input` listens for input events and updates the state.
- `:value` is a **binding** that sets the value of the input.



# Form V2

```
<script setup lang="ts">
import { ref } from "vue";

const name = ref("");
function handleClick() {
  name.value = "";
}
</script>

<template>
  <input v-model="name" placeholder="Type your name" />
  <p>Hello, {{ name }}</p>
  <button @click="handleClick">Submit</button>
</template>
```

- `v-model` is a **directive** that creates two-way binding between the input and the state.

# List Rendering

```
<script setup lang="ts">
import { ref } from "vue";
const items = ref(["Item 1", "Item 2", "Item 3"]);
</script>

<template>
  <ul>
    <li v-for="(item, index) in items" :key="index">
      {{ index + 1 }} - {{ item }}
    </li>
  </ul>
</template>
```

# Directives

- `v-for` is a **directive** that loops over an array and renders the template for each item.
- `:key` is a **binding** that tells Vue how to track each item in the list.
  - Without `:key`, Vue will re-render the entire list when the array changes, which is inefficient.

# Conditional Rendering

```
<script setup lang="ts">
import { ref } from "vue";
const toggle = ref(true);

function handleClick() {
  toggle.value = !toggle.value;
}
</script>

<template>
  <button @click="handleClick">Toggle</button>
  <div v-if="toggle">
    <p>Hello, World!</p>
  </div>
  <div v-else>
    <p>Goodbye, World!</p>
  </div>
</template>
```

# Directives

- `v-if` is a **directive** that conditionally renders the template based on the value of the expression.
- `v-else` is a **directive** that renders the template if the previous `v-if` is false.
- `v-else-if` is a **directive** that renders the template if the previous `v-if` is false and the expression is true.



# Todo App

- Create `todos` list using `ref([])`.
- Add new item from an input box.
- Show list with `v-for`.
- Prevent adding empty text.
- [Code](#)